HARP-PAF | Classroom Presentation Run Guide

Step-by-step instructions for presenting the already-built local application using Windows, Ubuntu (WSL), and VS Code. Keep this guide open during your demonstration.

START HERE: You need TWO running terminals - one for FastAPI (backend), and one for Vite/React (frontend). Do not close either while demonstrating.

1. Before leaving for class (recommended)

Use the same laptop where you developed and tested HARP-PAF. Confirm Ubuntu/WSL works, and that VS Code opens the folder /home/jimk7521/harp-paf. Have the four C++ example files ready. Charge your laptop, and open this PDF offline.

Your code is also pushed to the GitHub jim branch: https://github.com/tejas200713/harp-paf/tree/jim. GitHub is a backup and code viewer; you do not need internet to run the already-installed local project.

2. Open the project in VS Code

Where: Windows Start menu -> open Ubuntu terminal. Then type:

```
cd ~/harp-paf
code .
```

Wait for VS Code to open. You should see WSL: Ubuntu in the status bar and the harp-paf folder open. After VS Code opens, you can close the separate Ubuntu terminal. Keep VS Code open.

3. Start the backend (Terminal 1)

Where: VS Code menu -> Terminal -> New Terminal. Paste these commands, pressing Enter after each line:

```
cd ~/harp-paf
source venv/bin/activate
uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

Leave this terminal running. It should show a message similar to Uvicorn running on http://127.0.0.1:8000. The terminal will look busy and may not show a new shell prompt - that is correct.

Quick backend check: open a browser tab at http://127.0.0.1:8000/health. A successful response means the API is reachable.

4. Start the frontend (Terminal 2)

Where: VS Code Terminal panel -> click the + button to create a second terminal. Do not stop Terminal 1.

```
cd ~/harp-paf/frontend
npm run dev
```

Leave Terminal 2 running. Vite should display a Local: address, typically http://localhost:5173/. If the port is already in use, use the exact Local URL that Vite displays.

5. Open the application

Where: Windows browser (Chrome/Brave/Edge). Open http://localhost:5173/ (or the Local URL shown in Terminal 2). Keep both VS Code terminals running behind the browser.

Presentation walkthrough

Suggested live demonstration: about 5-8 minutes, plus questions.

6. What to say and click during the demo

A. Introduction (30-45 seconds)
"HARP-PAF is a local research prototype that analyzes four approved C++ workloads, uses workload features and a pairing heuristic to select task pairs, and compares measured execution times against a baseline. The frontend is React/Vite, and the backend is Python/FastAPI."

B. Home / Overview
Open the landing page and explain the workflow: choose the approved tasks, submit the job, wait for compilation and measurements, and inspect the results.

C. Upload / Create Job
Go to Upload (http://localhost:5173/upload). Choose exactly these source files from /home/jimk7521/harp-paf/examples/:

```
task1.cpp
task2.cpp
task3.cpp
task4.cpp
```

Submit the job and wait for processing to finish. The backend accepts the four approved source files, not arbitrary .cpp programs. If the file picker opens in Windows paths, browse to \\wsl.localhost\Ubuntu\home\jimk7521\harp-paf\examples (the distro display name may vary). Alternatively, use VS Code Explorer to locate the files.

D. Results
Open Results and show the baseline runtime, selected-pair runtime, measured speedup, and percentage runtime difference. Explain that these are actual local benchmark measurements, and the numbers can vary across runs and machines.

E. Pairs / Tasks / Predictions
Show which tasks were paired and the task-level features. Distinguish heuristic pairing estimates from measured benchmark outputs; unavailable prediction metrics or hardware counters should not be described as measured.

F. History / System
Open History to show saved runs, and System to show the detected WSL environment. You can revisit the latest completed run without rerunning everything.

7. If the live run takes time or fails

Use the History page to open a previously completed run. This is your safest backup during a short classroom demonstration. You can still show the result dashboard, task information, and pairing screen without starting a new benchmark.

Previous observed measurements were around a few milliseconds, so do not claim that a small speedup proves statistically significant performance gains. Also do not claim verified physical SMT core topology or hardware performance-counter measurements: the prototype was tested with WSL logical CPUs.

8. Which window should stay open?

Window	Purpose	Keep open?
VS Code Terminal 1	FastAPI backend / port 8000	YES
VS Code Terminal 2	Vite frontend / usually port 5173	YES
Browser	Your demonstration UI	YES
Separate Ubuntu terminal	Only used to open VS Code	Optional / may close

Troubleshooting & shutdown

9. If something does not open

Page says connection refused / cannot reach localhost: Check that both VS Code terminals are still running. Restart the stopped service using Section 3 or 4.

Frontend shows errors or blank page: Check Terminal 2 for errors, then refresh the browser with Ctrl+R. Check Terminal 1 if API requests are failing.

Port 5173 is already occupied: Vite may choose a different port, such as 5174. Open the exact URL shown by Vite instead of assuming 5173.

Port 8000 is already in use: An earlier backend may still be running. Find its terminal and stop it with Ctrl+C, then run the backend command again. Keep port 8000 because the frontend expects that backend URL.

Python says uvicorn is not found: In Terminal 1, ensure the venv was activated with source venv/bin/activate and that the shell prompt typically shows (venv).

npm command not found: Verify you opened VS Code in WSL: Ubuntu, not a Windows PowerShell terminal; Node/npm were used in the existing Ubuntu setup.

File selection fails: Make sure you selected the four original .cpp files, not the compiled files named task1, task2, etc. The backend validates the approved source contents.

Switching to a different laptop: The two launch commands alone will not be enough. Another laptop needs the repository checked out at branch jim, dependencies installed, compatible build tools, and a Linux/WSL environment. Test that setup beforehand rather than attempting it live.

10. After the presentation - stop everything correctly

Terminal 2 (frontend): click its terminal tab and press Ctrl+C.

Terminal 1 (backend): click its terminal tab and press Ctrl+C.

If the Python virtual environment is still active, you may type deactivate. Then close VS Code and the browser. Your code and saved benchmark artifacts remain on disk.

11. One-minute command cheat sheet

VS Code Terminal 1 - BACKEND:

```
cd ~/harp-paf
source venv/bin/activate
uvicorn backend.app:app --host 127.0.0.1 --port 8000
```

VS Code Terminal 2 - FRONTEND:

```
cd ~/harp-paf/frontend
npm run dev
```

Browser:

```
http://localhost:5173/
http://127.0.0.1:8000/health
```

Finish: Press Ctrl+C once in each running terminal.